# Project 13.1 — DispatchGuard-Multi-Agent-Outgoing-Message-Review-In-One-Pass

**A practice twin of Project 13 — CodeGuard.** Same skills — a Supervisor that **routes** work to only the specialist reviewers that apply, three narrow reviewers (one deterministic scanner, one **grounded in a real document via RAG**, one heuristic), and a **compile** step that merges findings into one de-duplicated, severity-sorted result. A **new, real-world job**. **There are no steps here on purpose.** Read the story, build it yourself, and only open the hints or the solution if you get stuck.

**Type:** Multi-agent (1 Supervisor + 3 reviewers) · **Stack:** Python, LangGraph (`Command`), LangChain, RAG (Chroma), OpenAI SDK, Pydantic · **Level:** Advanced

> **Stuck, or want to check your work?** Full hints · Complete solution

## The Story — what you are going to build

Every day a company sends outgoing messages to customers: marketing blasts, order receipts, internal memos that occasionally get forwarded out. One bad message — a leaked email address, a "we guarantee results" claim legal would never approve, a sale with no end date, a blast with no unsubscribe line — costs more than the message was ever worth. A human can't eyeball every one, and nobody wants three separate tools stapled together.

You are going to build **DispatchGuard**: a Supervisor-led team of three specialist reviewers that reads an outgoing message and returns one clean verdict before it's sent.

- A **Safety reviewer** (a deterministic scanner) flags leaked PII (emails, card-like numbers), internal markers left in the text, and legally risky claims ("guaranteed", "risk-free", "I promise").
- A **Brand reviewer**, grounded in the company's **real** voice guide through RAG, flags wording that breaks the actual house style — not what a model guesses good tone is.
- A **Completeness reviewer** (a heuristic) flags a marketing promo with no end date, or any blast with no unsubscribe line.
- A **Supervisor** routes each message by its **type** — an *internal* memo only gets the Safety check; a *transactional* receipt gets Safety + Completeness but skips Brand; a *marketing* blast gets all three — then **compiles** every finding into one de-duplicated, severity-sorted review.

**The one lesson of this project:** the same Supervisor + specialists + RAG shape you built for code review (CodeGuard) fits a completely different problem. Routing to skip irrelevant reviewers and

compiling several opinions into one clear verdict is the reusable architecture — the domain is just a skin.

## How it works, from a message to one verdict (picture the whole machine)

Read this once and the whole project should form in your head.

A message comes in with a leading type tag ( `[internal]` , `[transactional]` , `[marketing]` ). A **parser** splits off the type and the body. The **Supervisor** calls `choose_reviewers(message)` — routing by type: *internal* → Safety only; *transactional* → Safety + Completeness; *marketing* → all three. It then hands control to each chosen reviewer in turn.

Each reviewer does one narrow job and returns `Finding` s. **Safety** runs regex patterns over the body (deterministic, so it's testable, including on a clean message). **Brand** retrieves the relevant rules from the real voice guide (RAG over Chroma) and asks the model which lines break which retrieved rule — citing the actual rule, not general knowledge. **Completeness** applies simple heuristics (promo → needs an end date; every blast → needs an unsubscribe line).

When every chosen reviewer has run, the Supervisor's **compile** step de-duplicates findings by (category, message), sorts them by severity (high → medium → low), and formats one comment — or "cleared to send." if there's nothing. The reviewers run inside a LangGraph graph ( `START → supervisor` ; each reviewer returns `Command(goto="supervisor", ...)` ; the supervisor routes to the next pending reviewer, then to `compile → END` ).

That's the whole machine: **one message → the Supervisor routes by type → the applicable reviewers run (one scanner, one RAG-grounded, one heuristic) → compile into one de-duplicated, severity-sorted verdict.**

## What you are building (the pieces)

- A **parser** ( `parse_message(text) -> {type, body}` ) and a `Finding` model (severity, category, message).
- Three reviewers: `run_safety(message)` , `run_brand(model, collection, message)` (RAG + LLM), `run_completeness(message)` .
- A **RAG layer** over a real voice guide ( `get_collection` , `build_voice_store` , `retrieve_rules` ).
- A **Supervisor** ( `choose_reviewers` , `decide_next` , `compile_review` ) and the LangGraph **graph**.

**Inputs:** an outgoing message with a type tag. **Outputs:** one de-duplicated, severity-sorted review, or "cleared to send." **Rules:** each reviewer stays narrow; Safety and Completeness are deterministic

(so they're testable); Brand only cites rules it actually retrieved; the Supervisor only routes and compiles — it never reviews.

## A real example to build and test against

- `[marketing] Hey! Our guaranteed sale is here — email me at bob@acme.com for 50% off.` → HIGH (leaked email), MEDIUM (risky "guaranteed" claim; promo with no end date; no unsubscribe line), plus a LOW Brand finding citing the "Hey" rule from your guide.
- `[internal] confidential roadmap — do not share` → only Safety runs; it flags the internal markers. Brand and Completeness are **skipped**.
- `[transactional] Your order #1001 shipped. Track it here: ... Unsubscribe: ...` → Safety + Completeness run, Brand is skipped; nothing to flag → "cleared to send."

## How to practice this

1. Read this story. **Don't open the solution yet.** (Review Project 13's CodeGuard and Project 3's RAG if hazy — this is the same shape.)
2. Plan it on paper: which reviewer owns which job, and exactly which reviewers each message type should run. Settle the routing table before you write code.
3. Build it yourself. A good order: the Safety scanner on a dirty *and* a clean message → the Brand reviewer with RAG → the Completeness heuristic → the Supervisor (routing + compile) and the graph. Your call.
4. Test it on the real example: a dirty marketing blast (all three), an internal memo (Safety only), a clean transactional message ("cleared to send").
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

## Why this project is useful (read this at the end)

"Combine several specialist checks into one clear verdict, and only run the ones that apply" is one of the most common shapes in real software — content moderation, compliance review, multi-criteria approvals, pre-send checks all have it. Building it a second time, in a domain with nothing in common with code review, is the proof that you learned the *architecture* (a routing Supervisor + narrow, grounded specialists + a compile step), not just one app. The deterministic reviewers are what you can actually test; RAG is what keeps the judgment honest; routing is what keeps it cheap. Build DispatchGuard and you can point that same shape at almost any "review this before it goes out" problem.

## Done when

- Each reviewer was proven on its own — including the Safety scanner on a **clean** message — before joining the team.
- The Brand reviewer cites a rule from your real, searched voice guide, not general knowledge.
- The Supervisor routes by message type — an internal memo never runs Brand or Completeness.
- The final output is one de-duplicated, severity-sorted review, not three reports stuck together.
- Your tests pass: parsing + routing, the Safety scanner (dirty + clean), compile (dedup + sort), and a live full review.

---

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.